基于 AI 的 5G 网络关键性能指标预测与异常检测系统

5G 实验环境搭建详细文档

free5GC + UERANSIM 端到端 5G SA 仿真环境

环境：Windows / WSL2 / VirtualBox / Ubuntu 22.04 LTS

核心组件：free5GC v4.2.3 / UERANSIM v3.3.0 / MongoDB / gtp5g

1. 文档说明

本文档记录 5G 仿真实验环境的完整搭建过程，包括 Windows 下 WSL2 开发环境、VirtualBox Ubuntu 虚拟机、free5GC 核心网、UERANSIM gNodeB/UE、网络配置、UE 注册、端到端连通性验证，以及实验环境固化与恢复流程。内容根据实际搭建过程中的命令、配置与返回结果整理。

1.1 最终环境架构

```
Windows Host
│
├─ WSL2 Ubuntu 22.04
│  └─ 主开发环境
│     ├─ Git / Python / Docker
│     ├─ NumPy / Pandas / SciPy
│     ├─ scikit-learn / PyTorch
│     ├─ XGBoost / LightGBM
│     ├─ KPI 数据处理 / 预测 / 异常检测
│     └─ Streamlit
│
└─ VirtualBox Ubuntu 22.04
   └─ 5G 网络实验环境
      ├─ free5GC VM：192.168.56.101
      │  ├─ AMF / SMF / UPF / ...
      │  ├─ MongoDB
      │  └─ gtp5g
      └─ UERANSIM VM：192.168.56.102
         ├─ gNodeB
         └─ UE
```

项目	配置/说明
WSL2	Ubuntu 22.04，承担 Python/AI/数据分析主开发环境
free5GC VM	Ubuntu 22.04，Host-Only IP：192.168.56.101
UERANSIM VM	Ubuntu 22.04，Host-Only IP：192.168.56.102
网络结构	网卡 1：NAT；网卡 2：Host-Only
UE 隧道地址	uesimtun0：10.60.0.1
最终验收	UE 经 uesimtun0 成功访问外网

2. Windows 下配置 WSL2 Ubuntu 开发环境

2.1 安装 Ubuntu 22.04

在 D 盘建立 WSL 安装目录，并安装 Ubuntu 22.04：

```
wsl --install -d Ubuntu-22.04 --location D:\1_WSL\Ubuntu-22.04
```

初始化设置：用户名 lemonlinux；实验环境密码 1234。初始化完成后进行健康检查：

```
lsb_release -a
uname -r
python3 --version
git –version
```

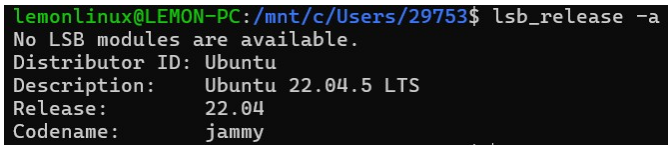


图 1 Ubuntu 22.04 环境验证

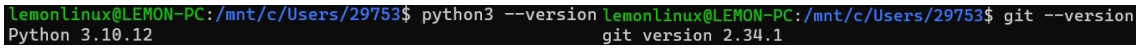


图 2 Python 与 Git 开发环境验证

2.2 更新系统并安装基础工具

```
sudo apt update
sudo apt upgrade -y
```

安装能够完成代码下载、编译、Python 管理和网络检查的基础工具：

```
sudo apt install -y \
git \
curl \
wget \
vim \
net-tools \
iproute2 \
build-essential \
cmake \
gcc \
g++ \
python3-pip \
python3-venv \
tcpdump
```

项目	配置/说明
git	下载和管理代码，后续 UERANSIM、free5GC 从 GitHub 获取
curl / wget	下载脚本、密钥、仓库配置等
vim	终端文本编辑器，用于修改 YAML 等配置文件
net-tools	提供 ifconfig、netstat 等网络命令
iproute2	提供 ip addr、ip route 等现代 Linux 网络工具
build-essential / gcc / g++	提供 C/C++ 编译环境
cmake	C/C++ 项目构建工具，UERANSIM 编译需要
python3-pip / venv	Python 包和虚拟环境管理

tcpdump	后续 5G 控制面/用户面抓包工具

3. WSL2 中安装 Docker

3.1 检查 systemd

```
ps -p 1 -o comm=
```

实际输出为 systemd，继续安装 Docker。

3.2 安装依赖并配置 Docker 官方仓库

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

3.3 安装与验收

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

docker --version
docker compose version

sudo usermod -aG docker $USER
newgrp docker

docker run hello-world
```

验收结果：返回 “Hello from Docker!”。

4. WSL 内核检查与实验架构调整

建立项目目录并检查内核、AVX、内核构建目录与已加载模块：

```
cd ~
mkdir -p ~/projects/5g-ai
cd ~/projects/5g-ai

echo "===== Kernel ====="
uname -r

echo "===== AVX ====="
grep -m1 -o 'avx[^ ]*' /proc/cpuinfo | head

echo "===== Kernel build directory ====="
ls -ld /lib/modules/$(uname -r)/build 2>/dev/null || echo "NO KERNEL BUILD DIR"

echo "===== Loaded modules ====="
lsmod | head
```

检查结果出现 “NO KERNEL BUILD DIR”。原因为当前环境使用 Microsoft WSL kernel，而非标准 Ubuntu generic kernel。考虑到 free5GC 用户面需要 gtp5g 内核模块，最终采用普通 Ubuntu VM 专门运行 5G 仿真，WSL2 保留为 AI 与数据分析主开发环境。

```
go version
# 返回：Command 'go' not found ...
```

5. VirtualBox Ubuntu 实验机搭建

5.1 创建基础虚拟机

下载 VirtualBox 7.2.18 Windows 版与 Ubuntu 22.04.5 Desktop AMD64 ISO，在 VirtualBox 中创建基础 Ubuntu 虚拟机：

项目	配置/说明
Name	5G-Base-Ubuntu
Location	D:\1_VM\5G-Lab
ISO	Ubuntu 22.04.5 Desktop
RAM	8 GB
CPU	4
Disk	60 GB

启动虚拟机并完成 Ubuntu 安装。随后检查系统、网络和内核构建目录，结果正常。



图 3 5G 实验基础 Ubuntu 虚拟机配置

5.2 安装 Guest Additions

```
sudo apt update
sudo apt install -y build-essential dkms linux-headers-$(uname -r)

ls /media/$USER/
cd /media/$USER/VBox_GAs_*
sudo ./VBoxLinuxAdditions.run
```

重启虚拟机，并启用共享双向剪贴板。

5.3 安装基础工具并制作母机

```
sudo apt update
sudo apt upgrade -y

sudo apt install -y \
build-essential \
cmake \
gcc \
g++ \
make \
git \
wget \
curl \
net-tools \
iproute2 \
iptables \
tcpdump \
linux-headers-$(uname -r)
```

版本检查：

```
git --version
gcc --version | head -1
g++ --version | head -1
cmake --version | head -1
make --version | head -1
tcpdump --version | head -1
```

基础环境验证成功后，将该虚拟机作为纯净母机并克隆：

项目	配置/说明
5G-Base-Ubuntu	母机，后续尽量不修改
free5gc	核心网 VM
ueransim	gNodeB + UE VM

6. 双虚拟机网络配置

6.1 双网卡结构

项目	配置/说明
网卡 1	NAT：用于 apt、git clone、软件下载安装及访问互联网
网卡 2	Host-Only：用于 free5GC VM 与 UERANSIM VM 之间通信

```
Windows Host
│
├─ free5gc VM
│   ├─ 192.168.56.101
│   └─ 5G Core：AMF / SMF / UPF / ...
│
└─ ueransim VM
    ├─ 192.168.56.102
    └─ RAN：gNodeB + UE
```

6.2 双向连通性测试

```
# ueransim VM
ping -c 4 192.168.56.101

# free5gc VM
ping -c 4 192.168.56.102
```

两方向均返回 4/4 received，0% packet loss，Host-Only 网络连通。

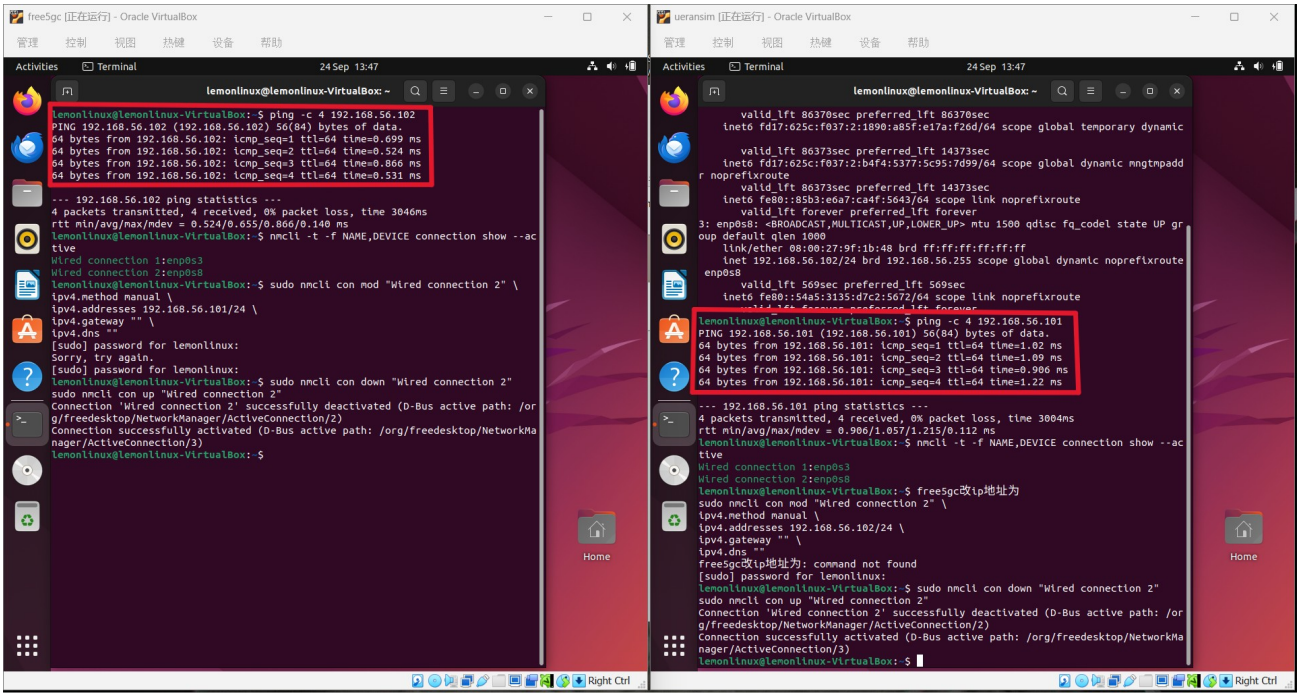


图 4 free5GC 与 UERANSIM 虚拟机双向网络连通测试

6.3 固定 Host-Only IP

先检查活动连接名称：

```
nmcli -t -f NAME,DEVICE connection show --active
```

Host-Only 接口为 enp0s8。free5gc VM 配置：

```
sudo nmcli con mod "Wired connection 2" \
ipv4.method manual \
ipv4.addresses 192.168.56.101/24 \
ipv4.gateway "" \
ipv4.dns ""
```

UERANSIM VM 配置：

```
sudo nmcli con mod "Wired connection 2" \
ipv4.method manual \
ipv4.addresses 192.168.56.102/24 \
ipv4.gateway "" \
ipv4.dns ""

sudo nmcli con down "Wired connection 2"
sudo nmcli con up "Wired connection 2"
ip addr show enp0s8
```

配置完成后再次互相 ping，网络连通正常。

7. 两台实验机公共依赖与 Go 环境

分别设置主机名：

```
# free5gc VM
sudo hostnamectl set-hostname free5gc

# ueransim VM
sudo hostnamectl set-hostname ueransim

sudo reboot
```

两台 VM 均安装公共依赖，并安装 Go 1.26.2：

```
sudo apt update
sudo apt install -y build-essential cmake gcc g++ make git net-tools iproute2 iptables tcpdump

cd /tmp
wget https://dl.google.com/go/go1.26.2.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.26.2.linux-amd64.tar.gz

mkdir -p ~/go/{bin,pkg,src}
echo 'export GOPATH=$HOME/go' >> ~/.bashrc
echo 'export GOROOT=/usr/local/go' >> ~/.bashrc
echo 'export PATH=$PATH:$GOPATH/bin:$GOROOT/bin' >> ~/.bashrc
source ~/.bashrc
```

检查：

```
hostname
go version
ip -4 addr show enp0s8
```

项目	配置/说明
free5gc VM	hostname: free5gc；Go: go1.26.2；IP: 192.168.56.101
UERANSIM VM	hostname: ueransim；Go: go1.26.2；IP: 192.168.56.102

公共环境阶段完成。

8. free5GC 核心网部署

8.1 安装 MongoDB

```
sudo apt install -y gnupg curl

curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc | \
sudo gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor

echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/8.2 multiverse" | \
sudo tee /etc/apt/sources.list.d/mongodb-org-8.2.list

sudo apt update
sudo apt install -y mongodb-org
sudo systemctl start mongod
sudo systemctl enable mongod
sudo systemctl status mongod
```

MongoDB 启动成功。

8.2 安装 Node.js 20 与 Yarn

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt update
sudo apt install -y nodejs
sudo corepack enable

node --version
npm --version
yarn --version
```

项目	配置/说明
Node.js 20.x	free5GC WebConsole 前端构建环境
corepack	管理 Yarn
Yarn	构建 free5GC WebConsole 前端

8.3 安装 gtp5g

```
cd ~
git clone https://github.com/free5gc/gtp5g.git
cd gtp5g
make
sudo make install

lsmod | grep gtp
```

gtp5g 内核模块验证成功。

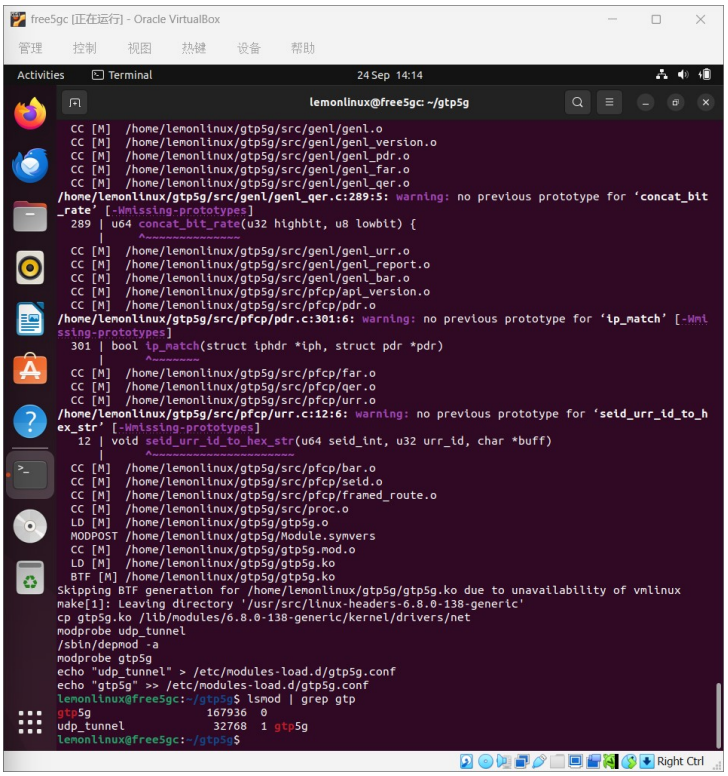


图 5 gtp5g 内核模块安装与加载成功

8.4 编译 free5GC v4.2.3 与 WebConsole

```
cd ~
git clone --recursive -b v4.2.3 -j $(nproc) https://github.com/free5gc/free5gc.git
cd ~/free5gc
make

make webconsole
ls ~/free5gc/webconsole/bin/
```

WebConsole 编译后返回 webconsole，可执行文件生成成功。

9. free5GC 网络配置

9.1 AMF N2/NGAP 配置

```
cd ~/free5gc
nano config/amfcfg.yaml

ngapIpList:
  - 192.168.56.101
ngapPort: 38412
```

检查：

```
grep -n -A3 "ngapIpList" config/amfcfg.yaml
```

9.2 SMF N3 配置

打开 config/smfcfg.yaml，查找 interfaceType: N3，将 N3 endpoint 从 127.0.0.8 修改为 192.168.56.101。

```
cd ~/free5gc
nano config/smfcfg.yaml

# 核心配置
- interfaceType: N3
  endpoints:
    - 192.168.56.101
  networkInstances:
    - internet

grep -n -A5 "interfaceType: N3" config/smfcfg.yaml
```

9.3 UPF GTP-U 配置

打开 config/upfcfg.yaml，查找 gtpu，将 N3 地址从 127.0.0.8 修改为 192.168.56.101：

```
cd ~/free5gc
nano config/upfcfg.yaml

gtpu:
  forwarder: gtp5g
  ifList:
    - addr: 192.168.56.101
      type: N3

grep -n -A15 "gtpu:" config/upfcfg.yaml
```

配置检查成功。

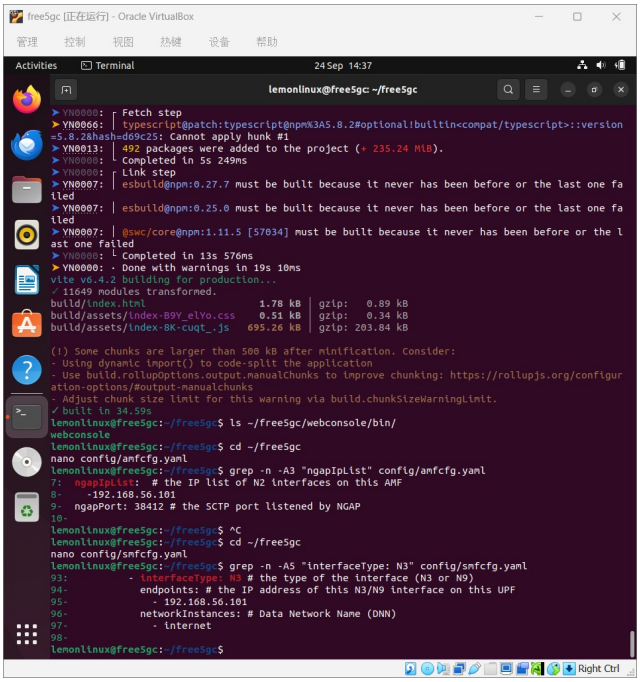


图 6 free5GC AMF、SMF 与 UPF 网络参数配置结果

10. UERANSIM 部署与配置

10.1 安装与编译

```
sudo apt update
sudo apt install -y libsctp-dev lksctp-tools
cmake --version

cd ~
git clone https://github.com/aligungr/UERANSIM
cd ~/UERANSIM
make

ls -lh ~/UERANSIM/build/
```

构建目录中生成 nr-binder、nr-cli、nr-gnb、nr-ue 等程序，编译成功。

```
total 14M
-rwxrwxr-x ... libdevbnd.so
-rwxrwxr-x ... nr-binder
-rwxrwxr-x ... nr-cli
-rwxrwxr-x ... nr-gnb
-rwxrwxr-x ... nr-ue
```

10.2 配置 gNodeB

编辑 ~/UERANSIM/config/free5gc-gnb.yaml，将 gNB 本地 N2/N3 地址与 AMF 地址修改为双 VM 网络地址：

```
nano ~/UERANSIM/config/free5gc-gnb.yaml

ngapIp: 192.168.56.102
gtpIp: 192.168.56.102

amfConfigs:
  - address: 192.168.56.101

grep -n -E "ngapIp|gtpIp|address:" ~/UERANSIM/config/free5gc-gnb.yaml
```

检查结果显示 ngapIp、gtpIp 均为 192.168.56.102，AMF address 为 192.168.56.101。

11. UE 用户参数与 WebConsole 注册

11.1 读取 UERANSIM UE 配置

```
cd ~/UERANSIM
grep -nE '^(supi|mcc|mnc|key|op|opType|amf):|^[[:space:]]+(sst|sd|apn|type):' config/free5gc-ue.yaml
```

项目	配置/说明
SUPI	imsi-208930000000001
MCC	208
MNC	93
Key	8baf473f2f8fd09487cccbd7097c6862
OP/OPC	8e27b6af0e692e750f32667a3b14605d
opType	OPC
AMF	8000
APN/DNN	internet
SST	0x01
SD	0x010203

11.2 启动 WebConsole 并创建 Subscriber

```
cd ~/free5gc/webconsole
./run.sh
```

WebConsole 启动后，在 Windows 主机浏览器访问：

```
http://192.168.56.101:5000
```

默认登录账号：Username = admin；Password = free5gc。登录成功后进入 SUBSCRIBERS，点击 CREATE，并按照 free5gc-ue.yaml 中的 UE 参数创建用户。

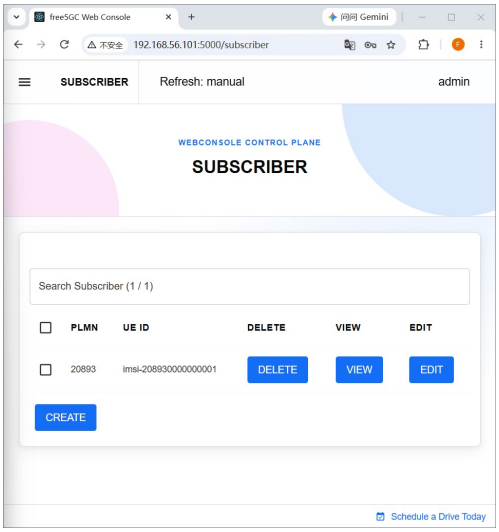


图 7 free5GC WebConsole 中 UE Subscriber 注册结果

12. 用户面转发、NAT 与端到端启动

12.1 临时启用 IP 转发与 NAT

已知 free5gc VM 的 NAT 网卡为 enp0s3。为使 UE 流量能够访问互联网，执行：

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE
sudo iptables -I FORWARD 1 -j ACCEPT
sudo systemctl stop ufw
```

返回 net.ipv4.ip_forward = 1。此阶段这些配置仍为临时配置，重启后可能需要重新执行。

```
lsmod | grep gtp5g
# 返回示例
gtp5g      167936  0
udp_tunnel   32768  1 gtp5g
```

12.2 启动 free5GC

```
cd ~/free5gc
./run.sh
```

终端持续输出核心网日志，free5GC 启动成功。

13. 故障排查：gNB 无法连接 AMF

13.1 故障现象

```
cd ~/UERANSIM
build/nr-gnb -c config/free5gc-gnb.yaml

UERANSIM v3.3.0
[sctp] [info] Trying to establish SCTP connection... (192.168.56.101:38412)
[sctp] [error] Connecting to 192.168.56.101:38412 failed. SCTP could not connect: Connection refused
```

初步检查发现 AMF 进程未正常运行。

13.2 单独启动 AMF 定位配置错误

```
cd ~/free5gc
./bin/amf -c ./config/amfcfg.yaml
```

AMF 返回 YAML 解析错误：

```
free5GC version: v4.2.3
go version: go1.26.2 linux/amd64
[ERRO][AMF][Main] AMF Run error: ReadConfig [./config/amfcfg.yaml] Error: [Factory] yaml: unmarshal errors:
  line 8: cannot unmarshal !!str `-192.16...` into []string
```

原因是修改 ngapIpList 时 YAML 列表格式错误。重新修正 amfcfg.yaml 后再次单独启动 AMF。

```
cd ~/free5gc
./bin/amf -c ./config/amfcfg.yaml

pgrep -a amf
sudo ss -lpn --sctp | grep 38412
```

验证结果：AMF 进程存在，并成功监听 192.168.56.101:38412。

```
17811 ./bin/amf -c ./config/amfcfg.yaml
LISTEN 0 128 192.168.56.101:38412 0.0.0.0:* users:(("amf",pid=17811,fd=7))
```

回到 UERANSIM VM 重新启动 gNB，连接成功。

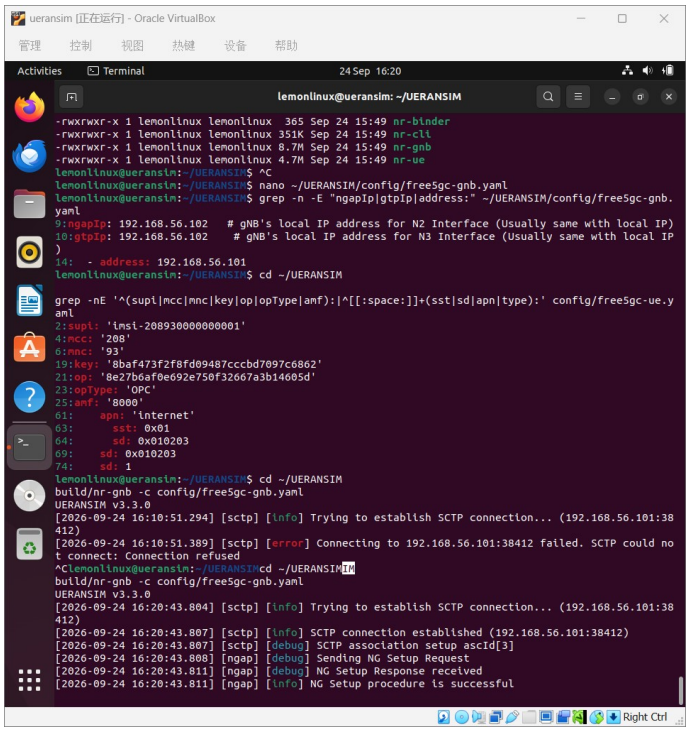


图 8 AMF 配置修复后 gNodeB 成功建立 SCTP/NGAP 连接

14. UE 启动与端到端连通性验证

14.1 启动 UE

```
cd ~/UERANSIM
sudo build/nr-ue -c config/free5gc-ue.yaml
```

新开终端检查 UE 隧道接口：

```
ifconfig uesimtun0

uesimtun0: flags=369<UP,POINTOPOINT,NOTRAILERS,RUNNING,PROMISC>  mtu 1400
        inet 10.60.0.1  netmask 255.255.255.0  destination 10.60.0.1
        RX packets 0  bytes 0 (0.0 B)
        TX packets 7  bytes 448 (448.0 B)
```

UE 获得隧道接口 uesimtun0，IPv4 地址为 10.60.0.1。

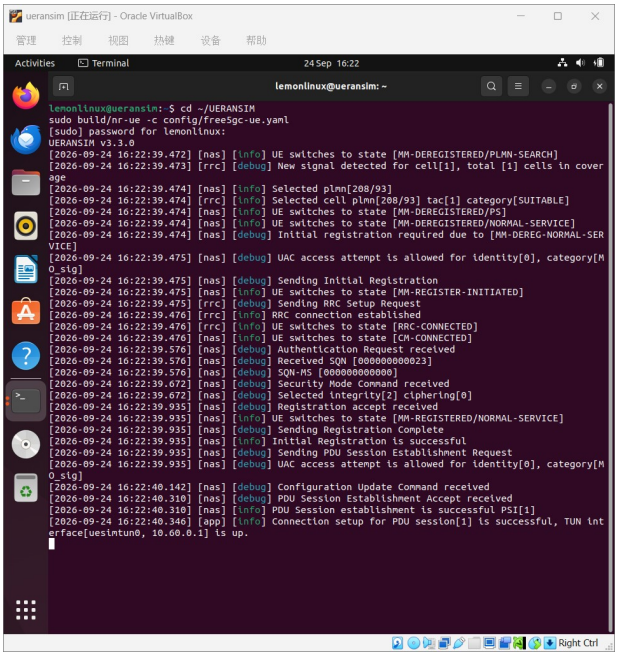


图 9 UE 注册及 PDU Session 建立成功

14.2 验证 UE 访问互联网

```
ping -I uesimtun0 google.com

PING google.com (142.250.140.100) from 10.60.0.1 uesimtun0: 56(84) bytes of data.
64 bytes from wj-in-f100.1e100.net (142.250.140.100): icmp_seq=1 ttl=62 time=12.2 ms
64 bytes from wj-in-f100.1e100.net (142.250.140.100): icmp_seq=2 ttl=62 time=10.8 ms
64 bytes from wj-in-f100.1e100.net (142.250.140.100): icmp_seq=3 ttl=62 time=9.87 ms
64 bytes from wj-in-f100.1e100.net (142.250.140.100): icmp_seq=4 ttl=62 time=10.8 ms
```

连续收到 ICMP Reply，说明 UE → gNodeB → free5GC UPF → NAT → Internet 的，环境构建成功。随后建立虚拟机快照作为备份。

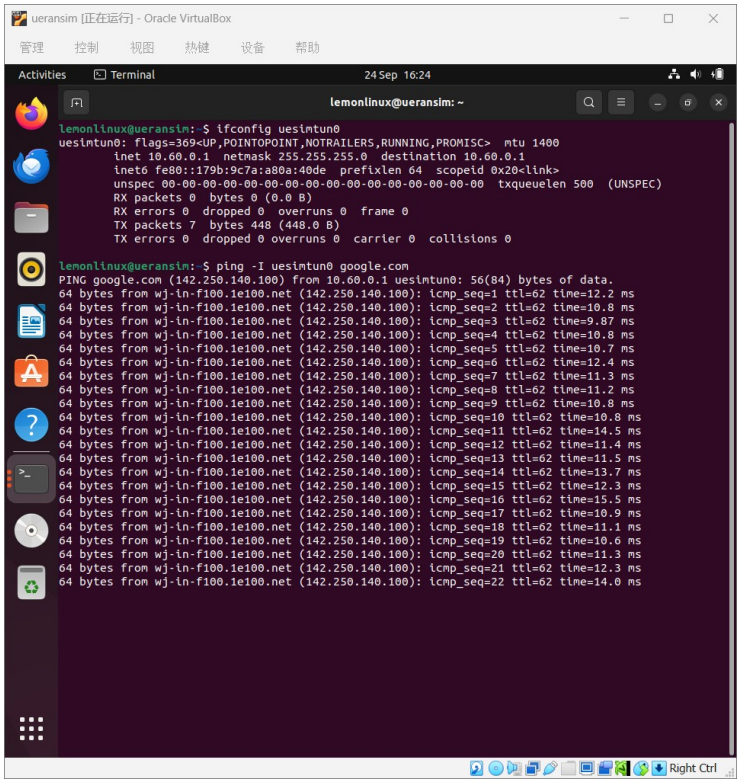


图 10 UE 经 uesimtun0 访问外部数据网络测试成功

15. 后续实验恢复流程

15.1 free5gc VM

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE
sudo iptables -I FORWARD 1 -j ACCEPT
sudo systemctl stop ufw

sudo systemctl start mongod
sudo modprobe gtp5g

cd ~/free5gc
./run.sh
```

需要打开 WebConsole 时另开终端：

```
cd ~/free5gc/webconsole
./run.sh
```

15.2 UERANSIM VM

```
# gNB
cd ~/UERANSIM
build/nr-gnb -c config/free5gc-gnb.yaml

# UE（另开终端）
cd ~/UERANSIM
sudo build/nr-ue -c config/free5gc-ue.yaml

# 最终检查
ip addr show uesimtun0
ping -I uesimtun0 google.com
```

16. 实验环境固化

16.1 永久启用 IP Forwarding

```
sudo tee /etc/sysctl.d/99-free5gc.conf > /dev/null <<'EOF'
net.ipv4.ip_forward=1
EOF

sudo sysctl --system
sysctl net.ipv4.ip_forward
```

验证返回 net.ipv4.ip_forward = 1，重启后不再恢复为 0。

16.2 持久化 iptables NAT/FORWARD 规则

```
sudo iptables -t nat -S POSTROUTING
sudo iptables -S FORWARD

# 验证规则
-P POSTROUTING ACCEPT
-A POSTROUTING -o enp0s3 -j MASQUERADE
-P FORWARD ACCEPT
-A FORWARD -j ACCEPT

sudo apt update
sudo apt install -y iptables-persistent
sudo netfilter-persistent save

sudo grep -E "MASQUERADE|FORWARD" /etc/iptables/rules.v4
```

保存文件中存在 FORWARD ACCEPT 与 POSTROUTING MASQUERADE 规则。

16.3 永久关闭 UFW

```
sudo ufw disable
sudo systemctl disable --now ufw
sudo ufw status
```

返回 Status: inactive。

16.4 gtp5g 与 MongoDB 自动加载

```
cat /etc/modules-load.d/gtp5g.conf
# 返回
udp_tunnel
gtp5g

lsmod | grep gtp5g
# 返回
gtp5g      167936  0
udp_tunnel   32768  2 gtp5g,sctp

systemctl is-enabled mongod
# 返回
enabled
```

16.5 固化后的环境状态

项目	配置/说明
192.168.56.101 静态 IP	enabled / 永久
IP forwarding	enabled / 永久
iptables NAT	enabled / 永久
iptables FORWARD	enabled / 永久
UFW 关闭	enabled / 永久
gtp5g 自动加载	enabled / 永久
MongoDB 自动启动	enabled / 永久

17. 重启后的最简启动流程

17.1 free5gc VM

```
cd ~/free5gc
./run.sh

# 如需 WebConsole
cd ~/free5gc/webconsole
./run.sh
```

17.2 UERANSIM VM

```
cd ~/UERANSIM
build/nr-gnb -c config/free5gc-gnb.yaml

# 另开终端
cd ~/UERANSIM
sudo build/nr-ue -c config/free5gc-ue.yaml

# 最终验证
ping -I uesimtun0 google.com
```

18. 搭建结果总结

本实验完成了从基础开发环境到端到端 5G SA 仿真链路的完整搭建。最终采用 WSL2 与 VirtualBox 分工的方式：WSL2 作为后续 AI/KPI 数据处理开发环境，VirtualBox 中的两台 Ubuntu 22.04 虚拟机分别运行 free5GC 核心网与 UERANSIM gNodeB/UE。

- free5GC 核心网已完成编译、配置和启动。
- UERANSIM gNodeB 与 UE 已完成编译和配置。
- free5GC VM 与 UERANSIM VM 通过 Host-Only 网络互通。
- gNB 已通过 SCTP/NGAP 成功连接 AMF。
- UE 已成功注册并获得 uesimtun0 接口与 10.60.0.1 地址。
- UE 可通过 uesimtun0 成功访问互联网，端到端用户面验证通过。
- IP forwarding、iptables、UFW、gtp5g 与 MongoDB 已完成持久化配置。

因此，“部署 free5GC + UERANSIM 并完成端到端 5G 网络连接测试”的环境目标已经完成，可进入后续网络数据采集与协议分析阶段。